

Spring Scheduler

0. Spring Scheduler 은 어떤 친구인가?

- Spring Scheduler 는 특정 시점에 또는 일정 주기로 메소드를 자동으로 실행한다
- 별도의 외부 스케줄러 없이 애플리케이션 내부에서 실행이 가능하다

ex) 5초마다 method() 를 실행 매일 새벽 2시에 작업 돌리기

아직 많이 사용해보진 않았지만, 외부 API 연동 시 재시도 로직이나 미처 처리하지 못한 데이터에 대해 특정 시간대에 배치 작업을 돌리기 유용할 것 같아 한 번 알아보았다.

1. 이 친구는 어떻게 다뤄야 하나?

스케줄링 기능 켜기 : 설정 클래스에 `@EnableScheduling` 을 넣어줘야 한다.

```
@Configuration
@EnableScheduling // 이거 추가해줘야 함
public class SchedulerConfig {
    ...
}
```

- `@SpringBootApplication` 이 붙은 메인 클래스에 붙여도 된다
- 해당 어노테이션이 없으면 동작하지 않는다

2. 이 친구는 어떻게 실행되나?

```
@Component
public class MyScheduler {

    @Scheduled(fixedDelay = 5000) // 5초마다
    public void doTask() {
        ...
    }
}
```

- 메소드의 반환 타입은 **void**여야 하며, 값을 반환해도 무시된다
- 메소드는 파라미터가 없어야 한다

- 해당 클래스는 반드시 **Spring Bean** 이어야 한다
- Spring 이 관리하는 Bean 이어야 스캔 과정에서 메소드를 찾을 수 있다

3. 옵션 종류

fixedDelay, fixedRate, cron, initialDelay 등이 있다.

3.1 fixedDelay

- 이전 작업이 끝난 시점부터 지정 시간(ms) 뒤에 다음 작업을 실행한다
- 작업끼리 겹치지 않는다

```
@Scheduled(fixedDelay = 5000) // 작업 종료 -> 5초 대기 -> 다음 작업 시작
```

3.2 fixedRate

- 이전 작업이 시작한 시점부터 지정 시간(ms) 뒤에 다음 작업을 실행한다
- 작업이 오래 걸리면 겹치거나 밀릴 수 있다

```
@Scheduled(fixedRate = 5000) // 작업 시작 -> 5초 뒤 -> 다음 작업 시작
```

단, 기본 설정인 단일 스레드 환경에서는 작업이 겹치더라도 동시에 실행되지 않고 직전 작업이 끝날 때까지 대기한다.

3.3 cron

- 리눅스 cron 표현식으로 특정 시각에 실행한다
- Spring 의 cron 은 6자리이다 : 초 분 시 일 월 요일

```
1) @Scheduled(cron = "0 0 2 * * *") // 매일 새벽 2시 0분 0초  
2) @Scheduled(cron = "0 */5 * * * *") // 5분마다  
3) @Scheduled(cron = "0 30 9 * * MON-FRI", zone = "Asia/Seoul") // 평일 9시 30분
```

zone 속성으로 타임존 지정이 가능하며, 지정하지 않으면 서버의 기본 타임존을 따른다.

3.4 initialDelay

- 애플리케이션 기동 후 첫 실행까지의 지연 시간이다
- fixedDelay / fixedRate 와 함께 사용한다

```
@Scheduled(initialDelay = 10000, fixedDelay = 5000) // 기동 10초 후 첫 실행,  
이후 종료 후 5초마다
```

```
@Scheduled(fixedDelayString = "5000") // 문자열/프로퍼티로 주입 가능
```

`fixedDelayString`, `initialDelayString` 처럼 `String` 형태로 주입할 수 있으며, `${property}` 형태의 프로퍼티 값도 주입이 가능하다.

4. 내부 동작

4.1 전체 흐름

```
@EnableScheduling
|
|-- @Import(SchedulingConfiguration)
|-- SchedulingConfiguration
|       : BeanPostProcessor + 여러 라이프사이클 인터페이스 구현체
|
|-- 모든 빈 초기화 시점에 @Scheduled 메소드 스캔
|-- SchedulingConfiguration 에 작업 등록
|-- ApplicationContext 준비 완료 후 TaskScheduler 가 실행 시작
```

4.2 @EnableScheduling 이 하는 일

- 내부에 `@Import(SchedulingConfiguration.class)` 를 가진다
- `SchedulingConfiguration` 은 `SchedulingConfiguration` 를 빈으로 등록한다

`@EnableScheduling` 은 결국 `BeanPostProcessor` 를 컨테이너에 심으라는 스위치일 뿐이다. 이게 없으면 스캔 자체가 일어나지 않아 `@Scheduled` 가 무시된다.

4.3 BeanPostProcessor 가 메소드 등록

- `SchedulingConfiguration` 는 `BeanPostProcessor` 다
- 모든 Bean 이 초기화될 때마다 `postProcessAfterInitialization()` 이 호출된다
- 해당 빈의 메소드를 리플렉션으로 훑어 `@Scheduled` 가 붙은 메소드를 찾는다
- 찾으면 그 메소드 정보를 `SchedulingConfiguration` 에 등록한다

`@Scheduled` 는 (AOP 처럼) 프록시를 만들지 않는다. `BeanPostProcessor` 가 단지 "이 빈의 메소드를 이 주기로 실행하라" 라고 스케줄러에 등록만 하는 것이다.

따라서 호출의 주체는 외부 코드가 아니라 스케줄러 스레드다. 스케줄러는 등록 시점에 확보한 빈 참조로 메소드를 직접 호출한다.

4.4 등록과 실행이 분리되어 있는 이유

- `BeanPostProcessor` 가 작업을 찾는 시점에는 아직 다른 빈들이 다 초기화되지 않았을 수 있다

- 찾자마자 실행하면, 작업이 의존하는 다른 빈이 준비되기 전에 실행될 위험이 있다
- 그래서 `ScheduledTaskRegistrar` 에 모아두었다가, 컨테이너가 완전히 준비되었다는 신호인 `ContextRefreshedEvent` (`ApplicationListener` 로 수신) 시점에 `TaskScheduler` 로 실제 작업을 넘겨 실행한다

4.5 TaskScheduler : 실제 실행 주체

- 명시적으로 빈을 등록하지 않으면, 기본값으로 단일 스레드 `ThreadPoolTaskScheduler` 를 사용한다
- 내부적으로 JDK 의 `ScheduledExecutorService` 를 감싼 것이라, `fixedDelay` , `fixedRate` , `cron` 계산도 이 위에서 이루어진다

단일 스레드이므로 작업이 길어지면 다른 스케줄도 밀린다. 등록된 모든 `@Scheduled` 작업이 하나의 스레드를 공유하기 때문이다.

작업을 병렬로 처리하려면 별도의 `TaskScheduler` 빈을 등록하고 풀 사이즈를 늘려주면 된다.

```
@Bean
public TaskScheduler taskScheduler() {
    ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
    scheduler.setPoolSize(5);
    scheduler.setThreadNamePrefix("scheduled-task-");
    return scheduler;
}
```

정리

옵션	기준 시점	특징
<code>fixedDelay</code>	이전 작업 종료 시점	작업이 겹치지 않음
<code>fixedRate</code>	이전 작업 시작 시점	길어지면 밀리거나 대기
<code>cron</code>	특정 시각	표현식 기반, 타임존 지정 가능
<code>initialDelay</code>	기동 후 첫 실행 지연	<code>fixedDelay</code> / <code>fixedRate</code> 와 병행

`@EnableScheduling` → `BeanPostProcessor` 가 `@Scheduled` 스캔 → `ScheduledTaskRegistrar` 등록 → 컨테이너 준비 완료 후 `TaskScheduler` 실행, 이 흐름이 핵심이다.

기본은 단일 스레드이므로, 작업이 많거나 길다면 풀 사이즈 조정을 고려하는 것이 좋다.